

SQL Interview Questions & Answers

40 Questions covering SQL Theory and Query Writing

Part A — SQL Theory & Concepts

Q1. What is the difference between DELETE, TRUNCATE, and DROP?

Answer:

DELETE removes rows one at a time, is a DML command, can use a WHERE clause, fires triggers, and can be rolled back within a transaction.

TRUNCATE removes all rows at once by deallocating data pages, is DDL, cannot use WHERE, resets identity/auto-increment counters, and is minimally logged (faster).

DROP removes the entire table structure along with its data, indexes, and constraints from the database permanently.

Q2. Explain the different types of JOINS in SQL.

Answer:

INNER JOIN returns only matching rows from both tables.

LEFT (OUTER) JOIN returns all rows from the left table and matched rows from the right; unmatched right-side columns are NULL.

RIGHT (OUTER) JOIN is the mirror of LEFT JOIN.

FULL OUTER JOIN returns all rows from both tables, with NULLs where there is no match.

CROSS JOIN returns the Cartesian product of both tables.

SELF JOIN joins a table to itself, typically using aliases, to compare rows within the same table.

Q3. What is normalization? Briefly describe 1NF, 2NF, and 3NF.

Answer:

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

1NF (First Normal Form): each column holds atomic values and there are no repeating groups.

2NF: table is in 1NF and every non-key column depends on the whole primary key (removes partial dependency, relevant for composite keys).

3NF: table is in 2NF and no non-key column depends on another non-key column (removes transitive dependency).

Q4. What is the difference between a PRIMARY KEY and a UNIQUE KEY?

Answer:

A PRIMARY KEY uniquely identifies each row, does not allow NULL values, and a table can have only one primary key (which may be composite).

A UNIQUE KEY also enforces uniqueness but allows one NULL value (in most databases), and a table can have multiple unique keys.

Q5. What is a FOREIGN KEY and why is it used?

Answer:

A FOREIGN KEY is a column (or set of columns) in one table that references the PRIMARY KEY of another table. It enforces referential integrity, ensuring that a value in the child table must exist in the parent table, preventing orphaned records.

Q6. What is the difference between WHERE and HAVING?

Answer:

WHERE filters individual rows before any grouping/aggregation takes place and cannot contain aggregate functions.

HAVING filters groups after GROUP BY has been applied and is typically used with aggregate functions such as SUM, COUNT, or AVG.

Q7. What are ACID properties in the context of database transactions?

Answer:

Atomicity: a transaction is all-or-nothing; if any part fails the whole transaction is rolled back.

Consistency: a transaction takes the database from one valid state to another, preserving all defined rules and constraints.

Isolation: concurrent transactions do not interfere with each other's intermediate results.

Durability: once a transaction is committed, its changes persist even after a system failure.

Q8. What is the difference between a clustered index and a non-clustered index?

Answer:

A clustered index determines the physical storage order of data rows in the table; a table can have only one clustered index because data can be sorted only one way.

A non-clustered index is a separate structure that holds a sorted reference (pointer) to the actual data rows; a table can have multiple non-clustered indexes.

Q9. What is a stored procedure and what are its advantages?

Answer:

A stored procedure is a precompiled, reusable block of SQL statements stored in the database and executed with a call, optionally accepting parameters.

Advantages include improved performance (precompiled execution plan), reduced network traffic, reusability, centralized business logic, and enhanced security via controlled access.

Q10. What is the difference between a stored procedure and a function in SQL?

Answer:

A function must return a single value or table and can be used inline in a SELECT statement; it generally cannot perform DML operations that change data or use transaction control.

A stored procedure can return zero, one, or multiple values (via OUT parameters), can execute DML/DDI statements, manage transactions, and is called using CALL/EXEC rather than in an expression.

Q11. What are database indexes, and what are the trade-offs of using them?

Answer:

An index is a data structure (commonly a B-tree) that speeds up data retrieval by allowing the engine to avoid scanning the entire table.

Trade-offs: indexes speed up SELECT/read queries but slow down INSERT, UPDATE, and DELETE operations since indexes must also be updated, and they consume additional storage space.

Q12. What is the difference between UNION and UNION ALL?

Answer:

UNION combines the result sets of two or more SELECT queries and removes duplicate rows, which requires an internal sort/comparison and is therefore slower.

UNION ALL combines result sets and keeps all duplicate rows, making it faster since no de-duplication step is needed.

Q13. What are database transaction isolation levels?

Answer:

READ UNCOMMITTED: allows dirty reads (reading uncommitted changes from other transactions).

READ COMMITTED: prevents dirty reads; only committed data is visible.

REPEATABLE READ: prevents dirty and non-repeatable reads, but phantom reads may still occur in some databases.

SERIALIZABLE: the strictest level; transactions are fully isolated as if executed sequentially, preventing dirty reads, non-repeatable reads, and phantom reads.

Q14. What is a deadlock, and how can it be avoided?

Answer:

A deadlock occurs when two or more transactions hold locks that the other transactions need, causing all of them to wait indefinitely.

It can be reduced by accessing tables/rows in a consistent order across transactions, keeping transactions short, using appropriate isolation levels, and adding indexes to reduce lock scope.

Q15. What is a CTE (Common Table Expression) and why would you use one?

Answer:

A CTE is a temporary, named result set defined with the WITH clause that exists only for the duration of the query. It improves readability of complex queries and enables recursive queries (e.g., traversing hierarchical data such as an org chart).

```
WITH high_earners AS (  
    SELECT emp_id, name, salary  
    FROM employees  
    WHERE salary > 80000  
)  
SELECT * FROM high_earners ORDER BY salary DESC;
```

Part B — SQL Query Writing

Assume an **employees** table (emp_id, name, dept_id, salary, manager_id, hire_date) and a **departments** table (dept_id, dept_name) unless stated otherwise.

Q16. Write a query to find the second highest salary from the employees table.

Answer:

Using LIMIT/OFFSET (MySQL/Postgres) or a subquery with MAX works well here.

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Alternative using window function
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t WHERE rnk = 2;
```

Q17. Write a query to find the Nth highest salary (generic solution).

Answer:

DENSE_RANK() handles ties correctly, so employees with equal salaries share the same rank.

```
SELECT salary FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) ranked
WHERE rnk = :N; -- replace :N with the desired rank
```

Q18. Write a query to find duplicate rows in a table based on the 'email' column.

Answer:

Group by the column being checked and filter groups whose count is greater than 1.

```
SELECT email, COUNT(*) AS occurrences
FROM customers
GROUP BY email
HAVING COUNT(*) > 1;
```

Q19. Write a query to delete duplicate rows, keeping only one copy of each.

Answer:

Use a window function to number duplicates and delete all rows except the first occurrence.

```
DELETE FROM customers
WHERE customer_id NOT IN (
    SELECT MIN(customer_id)
    FROM customers
    GROUP BY email
);
```

Q20. Write a query to find employees who earn more than their manager.

Answer:

Self-join the employees table so each employee row is compared against its manager's row.

```
SELECT e.name AS employee, e.salary, m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q21. Write a query to find departments that have more than 5 employees.

Answer:

Group employees by department and filter with HAVING.

```
SELECT dept_id, COUNT(*) AS emp_count
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 5;
```

Q22. Write a query to list each employee along with their department name, including departments with no employees.

Answer:

A RIGHT JOIN (or LEFT JOIN with tables swapped) from departments to employees preserves departments even when there are no matching employee rows.

```
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
ORDER BY d.dept_name;
```

Q23. Write a query to calculate the running total of salaries ordered by hire_date.

Answer:

A window function with SUM() OVER an ordered frame produces a running total without needing a self-join or correlated subquery.

```
SELECT emp_id, name, hire_date, salary,
       SUM(salary) OVER (ORDER BY hire_date
                        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running_tot
FROM employees;
```

Q24. Write a query to find the top 3 highest-paid employees in each department.

Answer:

Use ROW_NUMBER() partitioned by department, ordered by salary descending, then filter.

```
SELECT * FROM (
    SELECT emp_id, name, dept_id, salary,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM employees
) ranked
WHERE rn <= 3;
```

Q25. Write a query to find employees who do not belong to any department.

Answer:

Use a LEFT JOIN and filter for NULL on the departments side, or NOT IN / NOT EXISTS.

```
SELECT e.emp_id, e.name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id
WHERE d.dept_id IS NULL;
```

Q26. Write a query to swap the values of two columns (e.g., swap 'M' and 'F' in a gender column) in a single UPDATE statement.

Answer:

A CASE expression evaluates all rows using the original values before any row is changed, so the swap happens correctly in one pass.

```
UPDATE students
SET gender = CASE
```

```

        WHEN gender = 'M' THEN 'F'
        WHEN gender = 'F' THEN 'M'
        ELSE gender
    END;

```

Q27. Write a query to find the total salary paid per department along with the department name, sorted from highest to lowest.

Answer:

Join employees to departments, aggregate with SUM, and order the result.

```

SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id
GROUP BY d.dept_name
ORDER BY total_salary DESC;

```

Q28. Write a query to find customers who placed orders in every month of 2025.

Answer:

Count distinct months per customer for orders in 2025 and filter where the count equals 12.

```

SELECT customer_id
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = 2025
GROUP BY customer_id
HAVING COUNT(DISTINCT EXTRACT(MONTH FROM order_date)) = 12;

```

Q29. Write a query to find the difference in salary between each employee and the next highest-paid employee (salary gap).

Answer:

LAG()/LEAD() window functions let you compare a row's value to the value of a neighboring row without a self-join.

```

SELECT name, salary,
       salary - LEAD(salary) OVER (ORDER BY salary DESC) AS gap_to_next
FROM employees
ORDER BY salary DESC;

```

Q30. Write a query using EXISTS to find all customers who have placed at least one order.

Answer:

EXISTS stops scanning as soon as it finds one matching row, which is often more efficient than an IN subquery on large tables since it does not need to build a full result set.

```

SELECT c.customer_id, c.name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

```

Q31. Write a query to find the average salary per department, but only for departments where the average salary exceeds the company-wide average salary.

Answer:

Compare each department's average against the overall average using a subquery inside HAVING.

```

SELECT dept_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY dept_id
HAVING dept_avg > (SELECT AVG(salary) FROM employees);

```

```
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Q32. Write a query to find employees hired in the last 90 days.

Answer:

Use CURRENT_DATE (or GETDATE() / NOW() depending on the RDBMS) with a date interval.

```
SELECT emp_id, name, hire_date
FROM employees
WHERE hire_date >= CURRENT_DATE - INTERVAL '90 days';
```

Q33. Write a query to pivot rows into columns, showing total sales per product for each quarter (Q1-Q4) in a single row per product.

Answer:

Conditional aggregation with CASE inside SUM turns row values into columns without needing a database-specific PIVOT operator.

```
SELECT product_id,
       SUM(CASE WHEN quarter = 'Q1' THEN amount ELSE 0 END) AS q1_sales,
       SUM(CASE WHEN quarter = 'Q2' THEN amount ELSE 0 END) AS q2_sales,
       SUM(CASE WHEN quarter = 'Q3' THEN amount ELSE 0 END) AS q3_sales,
       SUM(CASE WHEN quarter = 'Q4' THEN amount ELSE 0 END) AS q4_sales
FROM sales
GROUP BY product_id;
```

Q34. Write a query to find the longest consecutive streak of days a user logged in.

Answer:

Subtracting a row number from the date (via ROW_NUMBER) produces a constant 'group' value for consecutive dates, which can then be grouped to measure streak length.

```
WITH numbered AS (
    SELECT user_id, login_date,
           login_date - (ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date)) *
FROM logins
)
SELECT user_id, COUNT(*) AS streak_length, MIN(login_date) AS streak_start
FROM numbered
GROUP BY user_id, grp
ORDER BY streak_length DESC
LIMIT 1;
```

Q35. Write a query to find all products that have never been ordered.

Answer:

A LEFT JOIN with a NULL check on the orders side identifies products with no matching order, or equivalently NOT EXISTS could be used.

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

Q36. Write a query to update a column value using data from another table (correlated UPDATE).

Answer:

An UPDATE...FROM (Postgres/SQL Server syntax) lets you set a column based on a matching row in another table.

```
UPDATE orders o
```

```

SET customer_name = c.name
FROM customers c
WHERE o.customer_id = c.customer_id;

```

Q37. Write a query to find the median salary of employees.

Answer:

PERCENTILE_CONT(0.5) computes the median directly on databases that support it (Postgres, Oracle, SQL Server); it interpolates when there is an even number of rows.

```

SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS median_salary
FROM employees;

```

Q38. Write a query to rank products within each category by sales, without skipping ranks for ties (e.g., 1, 1, 2, 3 instead of 1, 1, 3, 4).

Answer:

DENSE_RANK() assigns the same rank to tied rows and does not leave gaps in the sequence, unlike RANK().

```

SELECT category_id, product_id, total_sales,
       DENSE_RANK() OVER (PARTITION BY category_id ORDER BY total_sales DESC) AS rnk
FROM product_sales;

```

Q39. Write a query to find customers whose total order value is in the top 10% of all customers.

Answer:

NTILE(10) splits customers into 10 equal-sized buckets ordered by total spend; bucket 1 holds the top 10% when ordered descending.

```

WITH totals AS (
  SELECT customer_id, SUM(amount) AS total_spent
  FROM orders
  GROUP BY customer_id
),
buckets AS (
  SELECT customer_id, total_spent,
         NTILE(10) OVER (ORDER BY total_spent DESC) AS decile
  FROM totals
)
SELECT customer_id, total_spent FROM buckets WHERE decile = 1;

```

Q40. Write a query to find the number of new customers acquired each month, compared to the previous month (month-over-month change).

Answer:

Aggregate new customers per month using their first order date, then use LAG() to bring the previous month's count into the same row for comparison.

```

WITH first_orders AS (
  SELECT customer_id, MIN(order_date) AS first_order_date
  FROM orders
  GROUP BY customer_id
),
monthly AS (
  SELECT DATE_TRUNC('month', first_order_date) AS month,
         COUNT(*) AS new_customers
  FROM first_orders
  GROUP BY DATE_TRUNC('month', first_order_date)
)

```

```
SELECT month, new_customers,  
       new_customers - LAG(new_customers) OVER (ORDER BY month) AS change_from_prev  
FROM monthly  
ORDER BY month;
```